SEKENKOS.ID

Manual Book

Panduan Pengguna Website Toko Barang Preloved

[URL WEBSITE]

Nama pencipta:

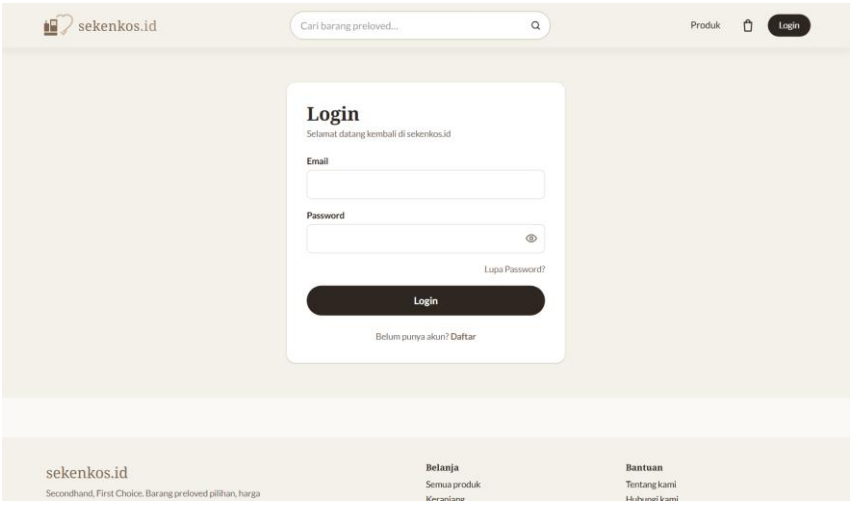
1. [Nama Dosen Pembimbing, Gelar] (NIP. [NIP Dosen])
2. [Nama Anggota 1] ([NIM Anggota 1])
3. [Nama Anggota 2] ([NIM Anggota 2])
4. [Nama Anggota 3] ([NIM Anggota 3])

Manual Book - sekenkos.id

1. LOGIN DAN PENDAFTARAN AKUN

1. Buka Halaman Login

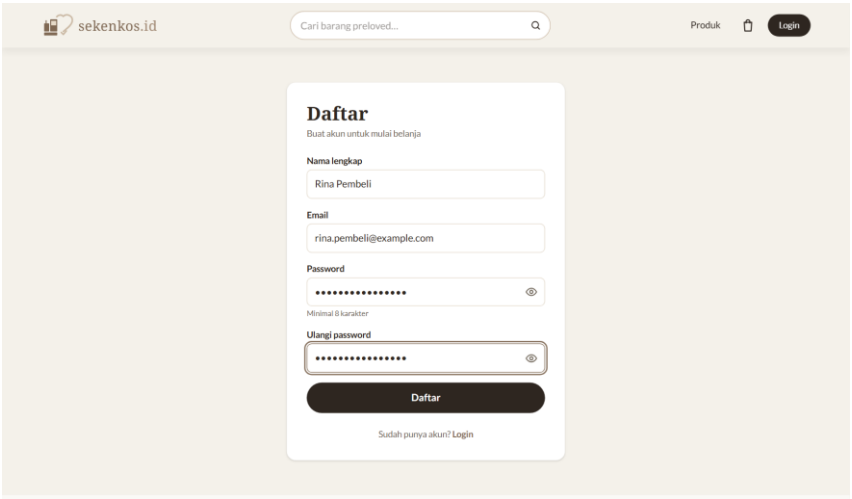
- Buka website sekenkos.id melalui browser, lalu klik tombol **Login** di pojok kanan atas.
- Halaman **Login** akan tampil dengan kolom **Email** dan **Password**.



Gambar 1.1 Halaman Login

2. Daftar Akun Baru

- Jika belum memiliki akun, klik tautan **Daftar** di bawah tombol **Login** (tulisan "Belum punya akun? Daftar").
- Isi **Nama lengkap**, **Email**, **Password** (minimal 8 karakter), dan **Ulangi password** dengan password yang sama.
- Klik tombol **Daftar**. Setelah berhasil, Anda langsung masuk ke akun baru.

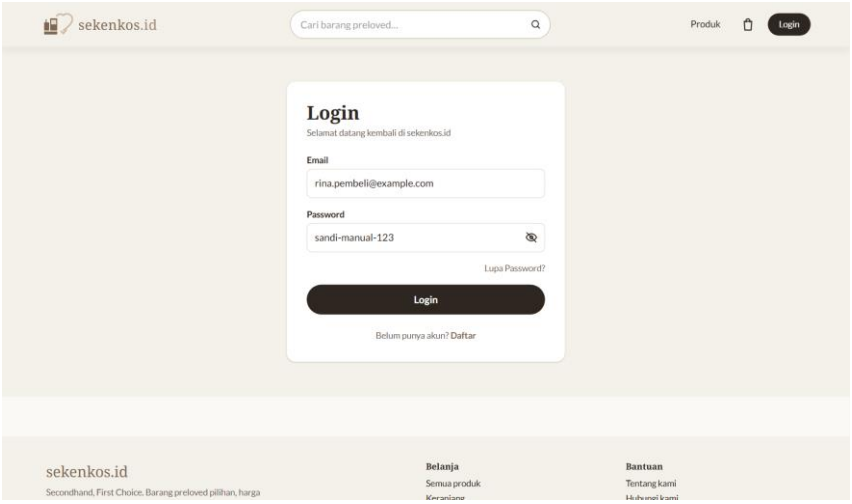


Gambar 1.2 Formulir pendaftaran akun

3. Masukkan Informasi Akun

- Masukkan **Email** dan **Password** yang sudah terdaftar.

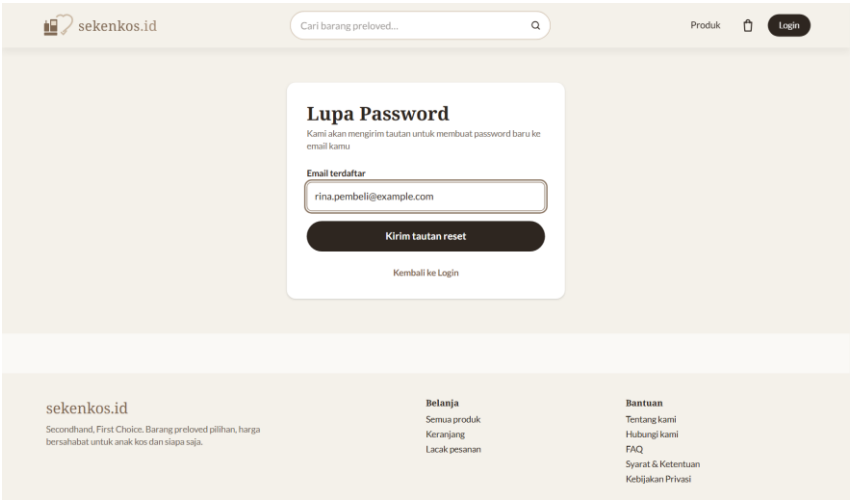
- Klik ikon mata di ujung kolom **Password** untuk menampilkan atau menyembunyikan kata sandi yang Anda ketik.



Gambar 1.3 Kolom email dan password yang sudah diisi

4. Lupa Password?

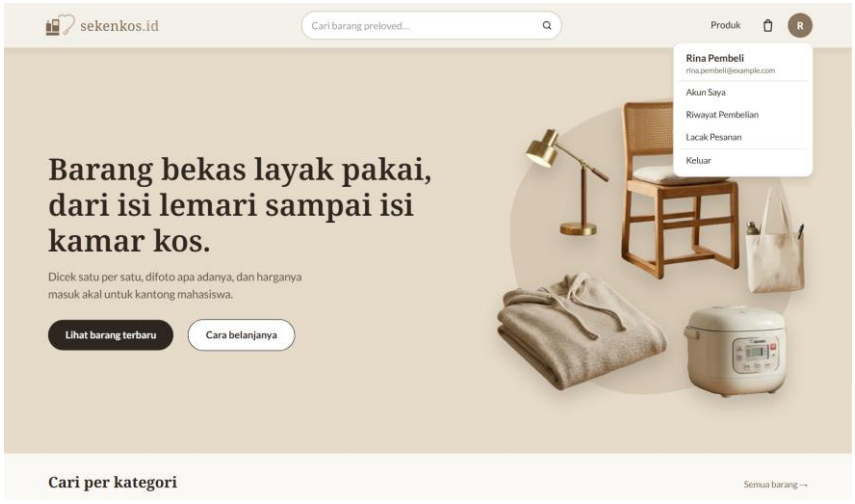
- Klik tautan **Lupa Password?** di halaman login.
- Masukkan **Email terdaftar**, lalu klik **Kirim tautan reset**.
- Buka email Anda dan klik tautan yang dikirim. Tautan hanya berlaku 1 jam dan hanya bisa dipakai sekali.
- Pada halaman **Password baru**, isi **Password baru** dan **Ulangi password baru**, lalu klik **Simpan password baru**.



Gambar 1.4 Halaman Lupa Password

5. Masuk ke Akun

- Klik tombol **Login**. Jika berhasil, tombol **Login** di pojok kanan atas berubah menjadi lingkaran berisi huruf awal nama Anda.
- Klik lingkaran tersebut untuk membuka menu **Akun Saya**, **Riwayat Pembelian**, **Lacak Pesanan**, dan **Keluar**.

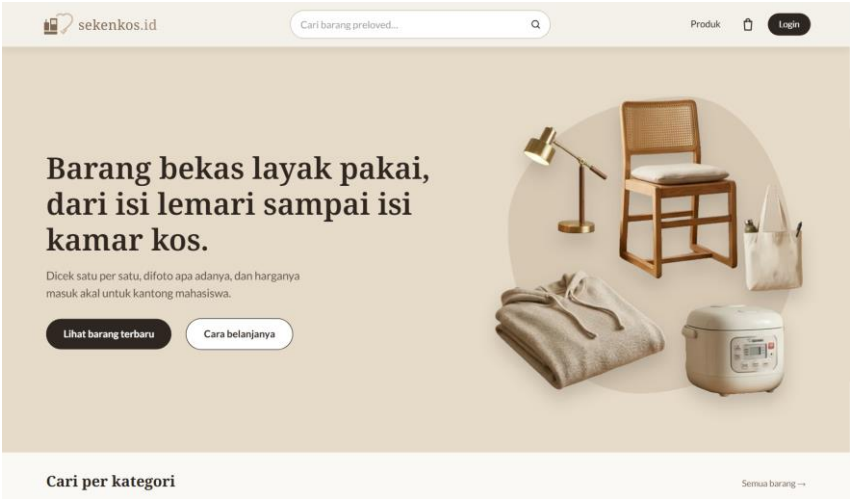


Gambar 1.5 Menu akun setelah berhasil masuk

2. MENJELAJAHI FITUR-FITUR WEBSITE

1. Halaman Utama

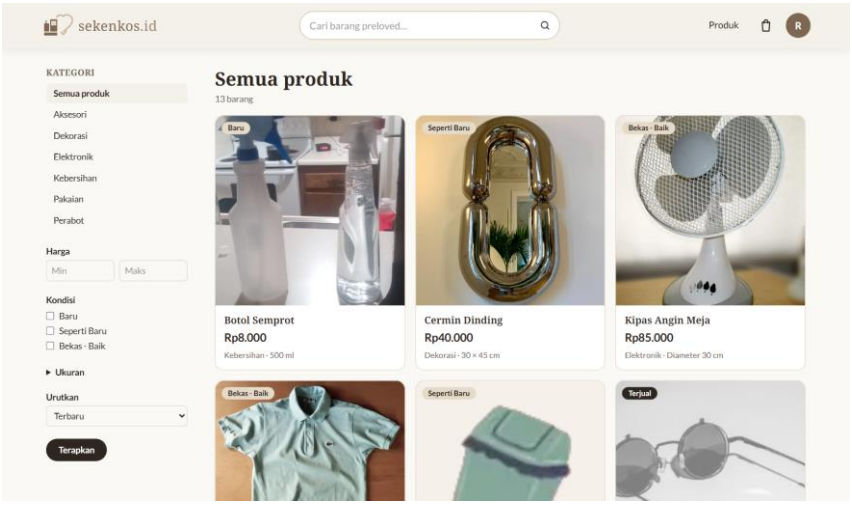
- Halaman utama menampilkan logo **sekenkos.id**, kolom pencarian, menu **Produk**, dan ikon keranjang belanja di bagian atas.
- Klik **Lihat barang terbaru** untuk melihat produk terbaru, atau **Cara belanjanya** untuk membaca panduan singkat berbelanja.
- Gulir ke bawah untuk melihat bagian **Cari per kategori**.



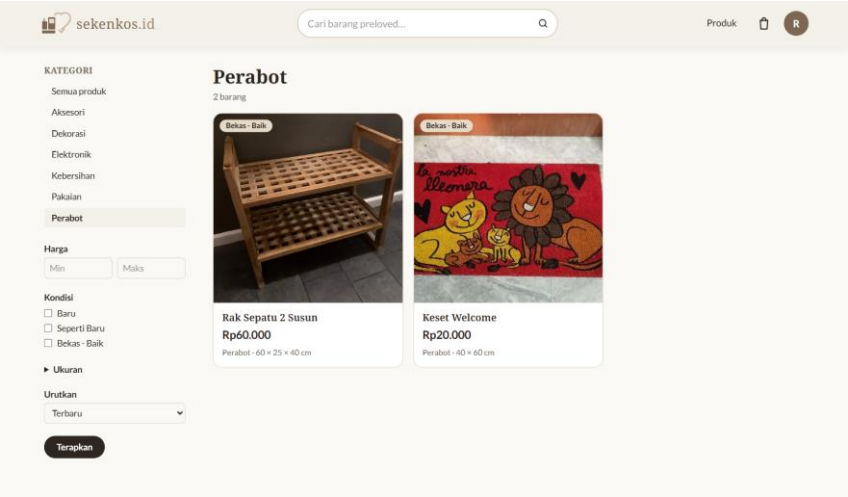
Gambar 2.1 Halaman utama sekenkos.id

2. Kategori Produk

- Klik menu **Produk** untuk membuka halaman **Semua produk**.
- Pilih kategori di panel **Kategori** sebelah kiri, misalnya **Aksesori**, **Dekorasi**, **Elektronik**, **Kebersihan**, **Pakaian**, atau **Perabot**.



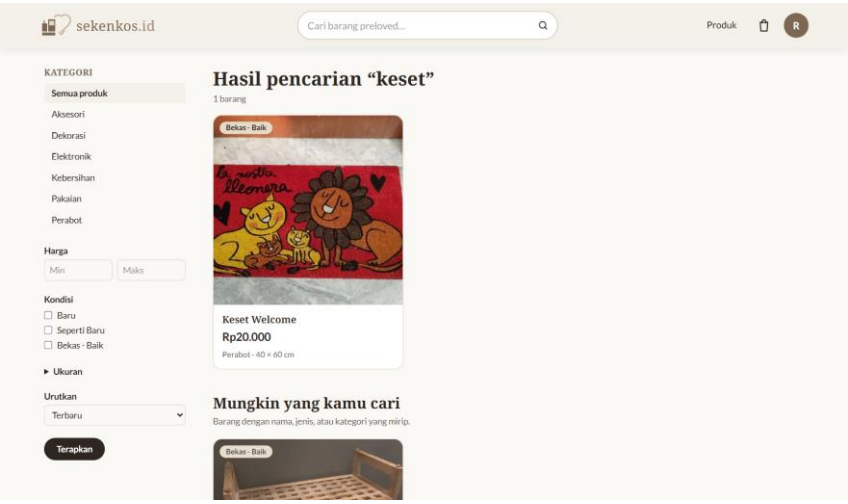
Gambar 2.2 Halaman Semua produk



Gambar 2.3 Daftar produk dalam kategori Perabot

3. Pencarian

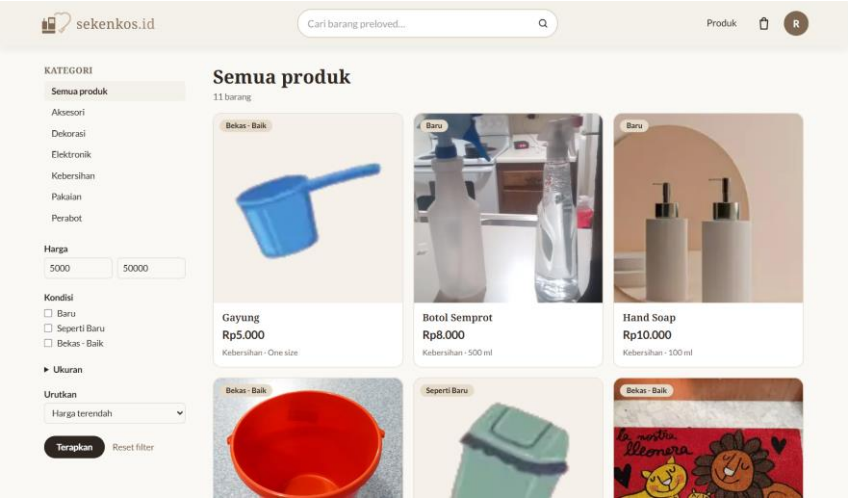
- Ketik nama barang pada kolom **Cari barang preloved…** di bagian atas, lalu tekan Enter.
- Hasil pencarian tampil di halaman **Hasil pencarian**. Di bawahnya terdapat bagian **Mungkin yang kamu cari** berisi barang serupa.



Gambar 2.4 Hasil pencarian kata "keset"

4. Filter dan Urutkan

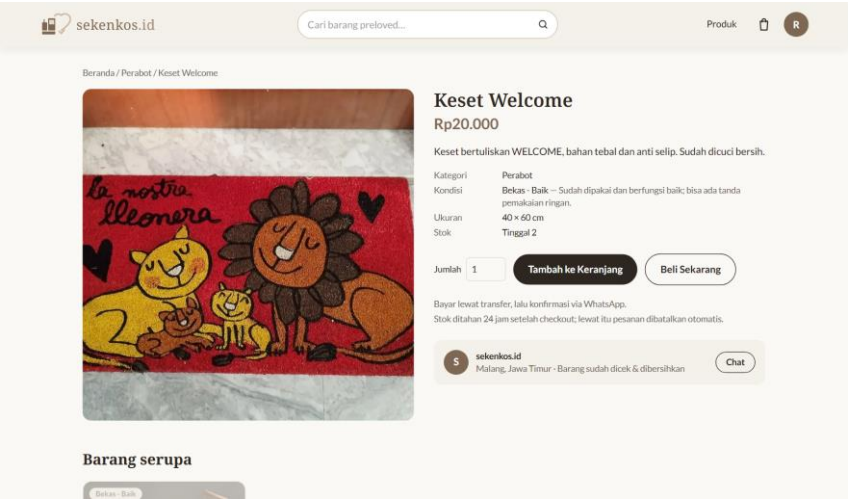
- Isi rentang **Harga** pada kolom **Min** dan **Maks**, lalu centang **Kondisi** barang yang diinginkan.
- Pilih **Urutan**: **Terbaru**, **Popularitas**, **Harga terendah**, atau **Harga tertinggi**.
- Klik **Terapkan** untuk menampilkan hasilnya.



Gambar 2.5 Filter harga dan urutan Harga terendah

5. Halaman Detail Produk

- Klik salah satu produk untuk membuka halaman detailnya.
- Halaman ini berisi foto, harga, deskripsi, **Kategori**, **Kondisi**, **Ukuran**, dan **Stok** barang.
- Klik **Chat** untuk bertanya langsung kepada penjual melalui WhatsApp. Di bagian bawah terdapat daftar **Barang serupa**.

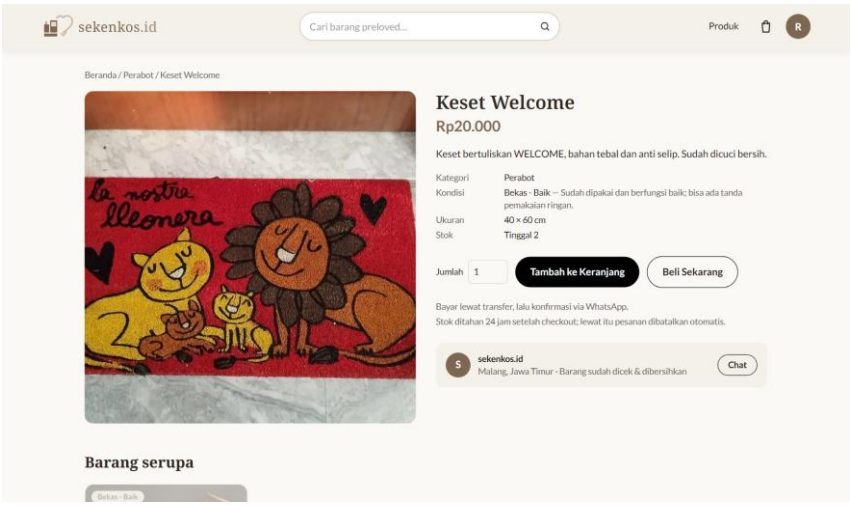


Gambar 2.6 Halaman detail produk

3. BELANJA DAN PEMBAYARAN

1. Tambah ke Keranjang

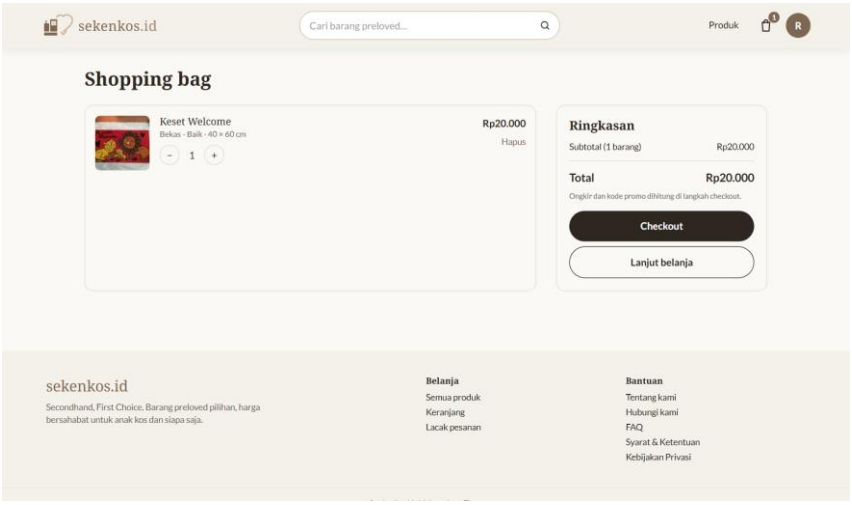
- Pada halaman detail produk, atur jumlah barang, lalu klik **Tambah ke Keranjang**.
- Angka pada ikon keranjang di pojok kanan atas akan bertambah. Klik **Beli Sekarang** jika ingin langsung ke halaman checkout.



Gambar 3.1 Barang berhasil ditambahkan ke keranjang

2. Melihat Keranjang

- Klik ikon keranjang di pojok kanan atas untuk membuka halaman **Shopping bag**.
- Ubah jumlah barang dengan tombol − dan +, atau klik **Hapus** untuk mengeluarkan barang dari keranjang.
- Periksa **Ringkasan** dan **Total**, lalu klik **Checkout**. Klik **Lanjut belanja** jika masih ingin mencari barang lain.

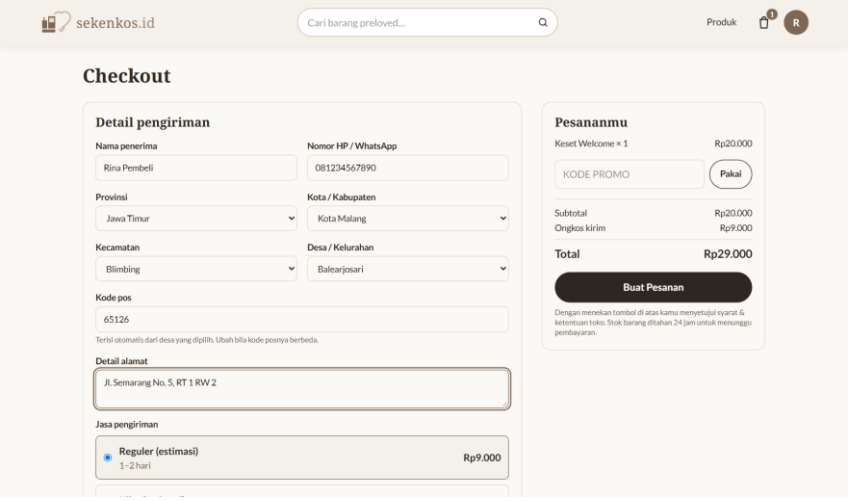


Gambar 3.2 Halaman keranjang belanja

3. Checkout

- Pada bagian **Detail pengiriman**, isi **Nama penerima** dan **Nomor HP / WhatsApp**.

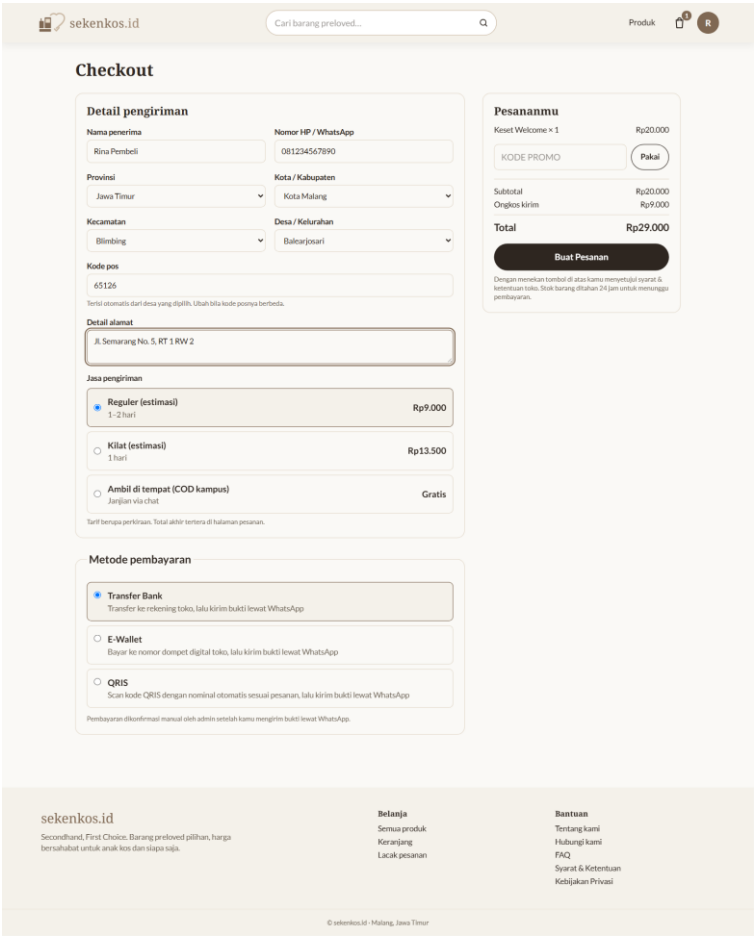
- Pilih **Provinsi**, **Kota / Kabupaten**, **Kecamatan**, dan **Desa / Kelurahan**. Kode pos akan terisi otomatis. Lengkapi **Detail alamat**.
- Pada **Jasa pengiriman**, pilih kurir dan layanan yang diinginkan. Ongkos kirim dihitung otomatis.
- Jika memiliki kode promo, ketik pada kolom **Kode promo** lalu klik **Pakai**.



Gambar 3.3 Formulir checkout

4. Pilih Metode Pembayaran

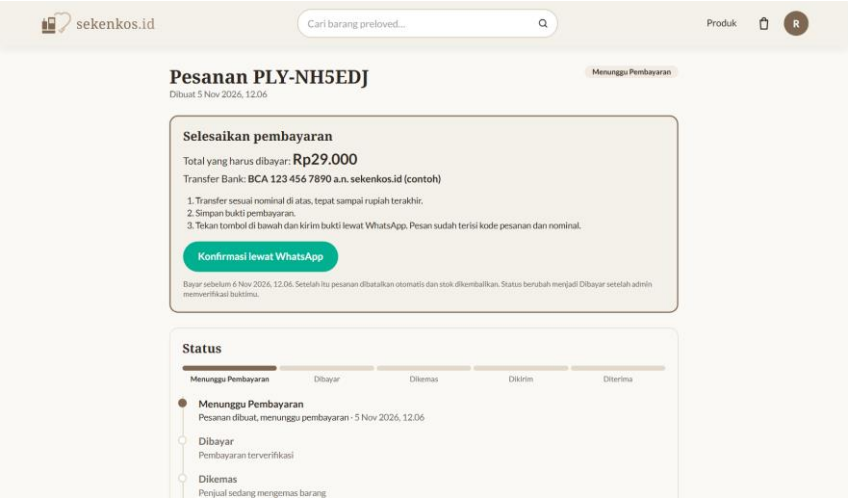
- Pada bagian **Metode pembayaran**, pilih **Transfer Bank**, **E-Wallet**, atau **QRIS**.
- Periksa ringkasan **Pesananmu**: **Subtotal**, **Ongkos kirim**, dan **Total**.
- Klik **Buat Pesanan**. Stok barang akan ditahan selama 24 jam untuk menunggu pembayaran Anda.



Gambar 3.4 Metode pembayaran dan ringkasan pesanan

5. Konfirmasi Pembayaran

- Setelah pesanan dibuat, halaman **Selesaikan pembayaran** menampilkan total tagihan dan tujuan pembayaran.
- Bayar sesuai nominal yang tertera, tepat sampai rupiah terakhir, lalu simpan bukti pembayaran.
- Klik **Konfirmasi lewat WhatsApp** dan kirim bukti pembayaran. Pesan WhatsApp sudah terisi kode pesanan dan nominal.
- Bayar sebelum batas waktu yang tertera. Jika lewat, pesanan dibatalkan otomatis. Status berubah menjadi **Dibayar** setelah admin memeriksa bukti Anda.

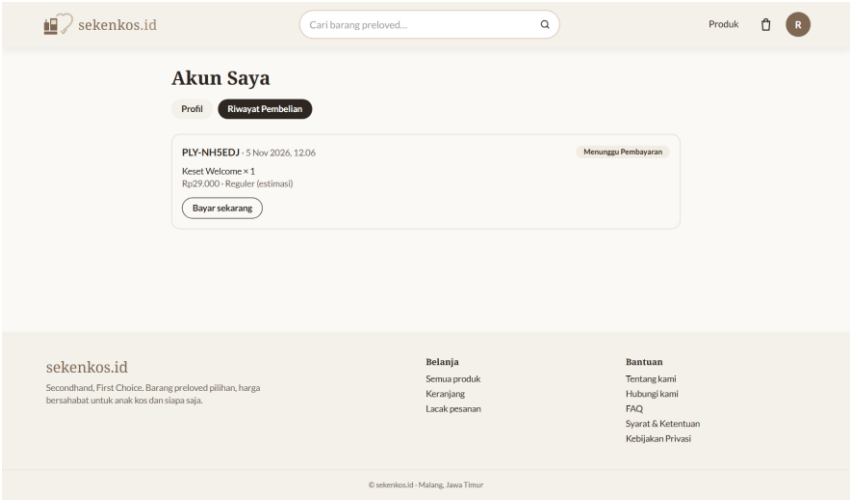


Gambar 3.5 Halaman Selesaikan pembayaran

4. MELACAK PESANAN

1. Riwayat Pembelian

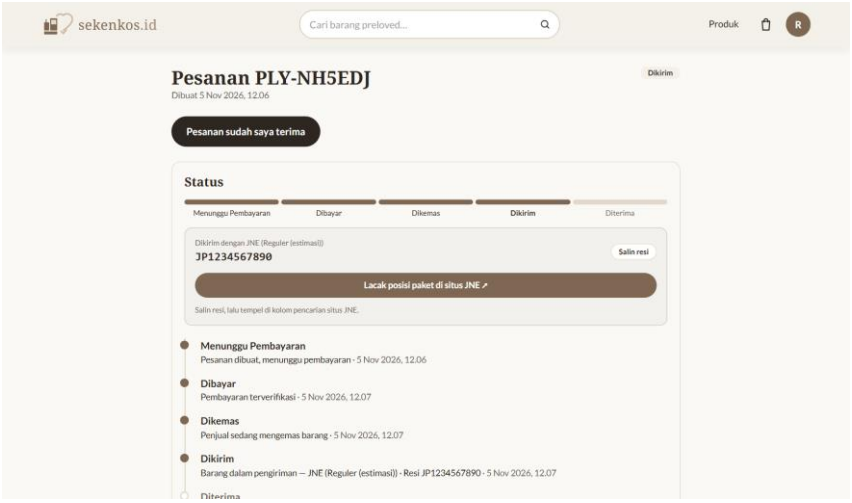
- Klik lingkaran inisial nama di pojok kanan atas, lalu pilih **Riwayat Pembelian**.
- Semua pesanan Anda tampil beserta statusnya. Klik **Bayar sekarang** untuk pesanan yang belum dibayar, atau **Detail & lacak** untuk melihat detail pesanan.



Gambar 4.1 Halaman Riwayat Pembelian

2. Memantau Status Pesanan

- Di halaman detail pesanan, garis kemajuan menunjukkan tahap pesanan: **Menunggu Pembayaran**, **Dibayar**, **Dikemas**, **Dikirim**, dan **Diterima**.
- Riwayat lengkap beserta tanggal dan jam tampil di bawahnya. Anda juga menerima email saat pesanan dibayar dan dikirim.



Gambar 4.2 Status pesanan yang sedang dikirim

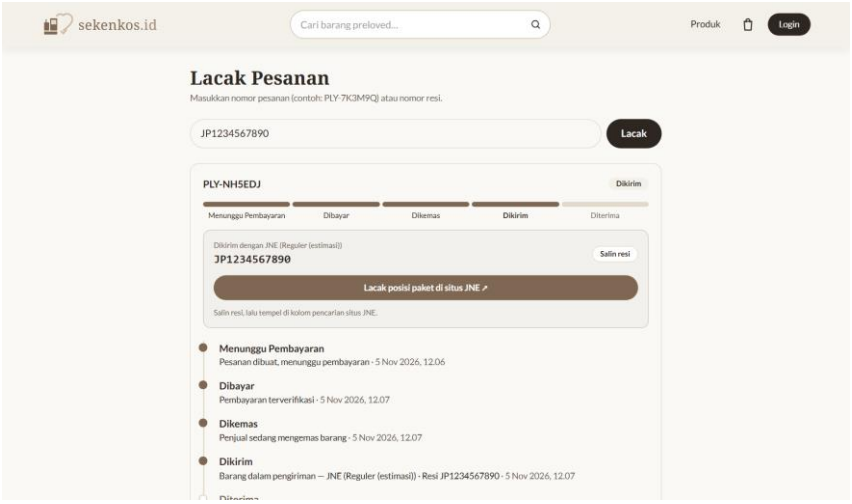
3. Lacak Posisi Paket di Situs Kurir

- Setelah pesanan dikirim, nama kurir dan nomor resi tampil di kotak pengiriman.
- Klik **Salin resi**, lalu klik **Lacak posisi paket di situs JNE** (nama kurir menyesuaikan pesanan Anda).

- Situs resmi kurir akan terbuka. Tempelkan nomor resi pada kolom pencarian untuk melihat posisi paket.

4. Lacak Tanpa Login

- Klik **Lacak pesanan** di bagian bawah halaman, atau pilih **Lacak Pesanan** dari menu akun.
- Masukkan nomor pesanan (contoh: PLY-7K3M9Q) atau nomor resi, lalu klik **Lacak**.



Gambar 4.3 Halaman Lacak Pesanan

5. Konfirmasi Pesanan Diterima

- Setelah paket sampai, buka detail pesanan dan klik **Pesanan sudah saya terima**.
- Status pesanan berubah menjadi **Diterima**.

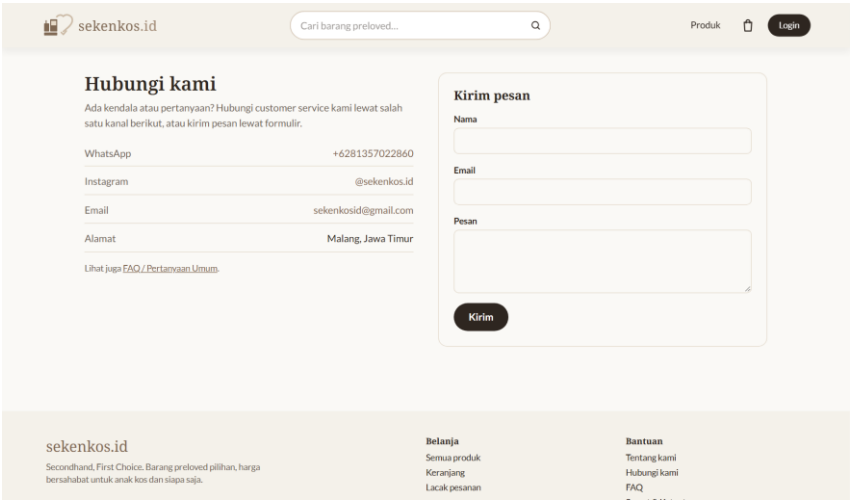
5. KONTAK DAN BANTUAN

1. Hubungi Customer Service

- Gulir ke bagian paling bawah halaman, lalu pada kolom **Bantuan** klik **Hubungi kami**.
- Halaman **Hubungi kami** berisi kontak **WhatsApp**, **Instagram**, **Email**, dan **Alamat** toko.
- Anda juga dapat mengirim pesan melalui formulir **Kirim pesan**: isi **Nama**, **Email**, dan **Pesan**, lalu klik **Kirim**.



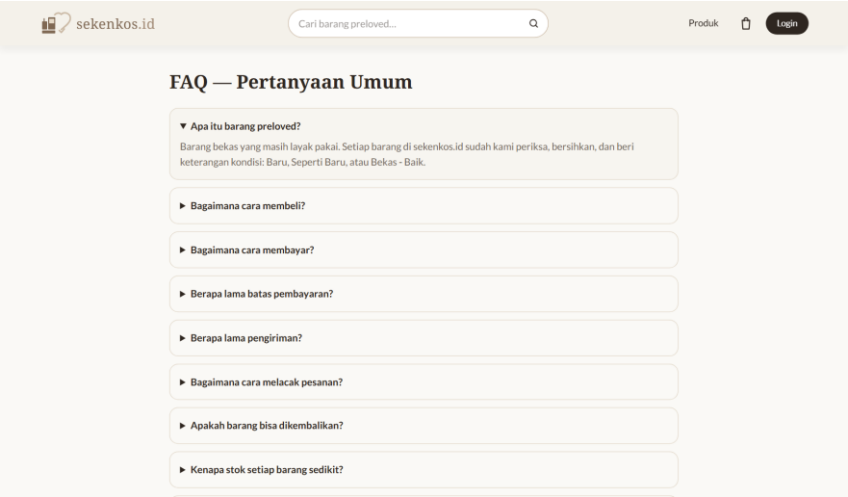
Gambar 5.1 Menu bantuan di bagian bawah halaman



Gambar 5.2 Halaman Hubungi kami

2. FAQ

- Pada kolom **Bantuan**, klik **FAQ** untuk membuka daftar pertanyaan yang sering diajukan.
- Klik sebuah pertanyaan untuk menampilkan jawabannya.



Gambar 5.3 Halaman FAQ

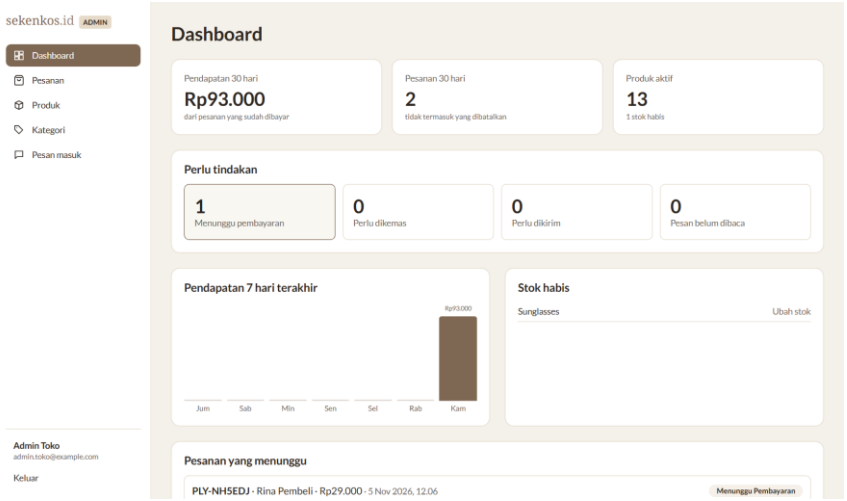
3. Informasi Lainnya

- Klik **Tentang kami**, **Syarat & Ketentuan**, atau **Kebijakan Privasi** pada kolom **Bantuan** untuk membaca informasi toko.

6. PANDUAN ADMIN

1. Masuk ke Dashboard Admin

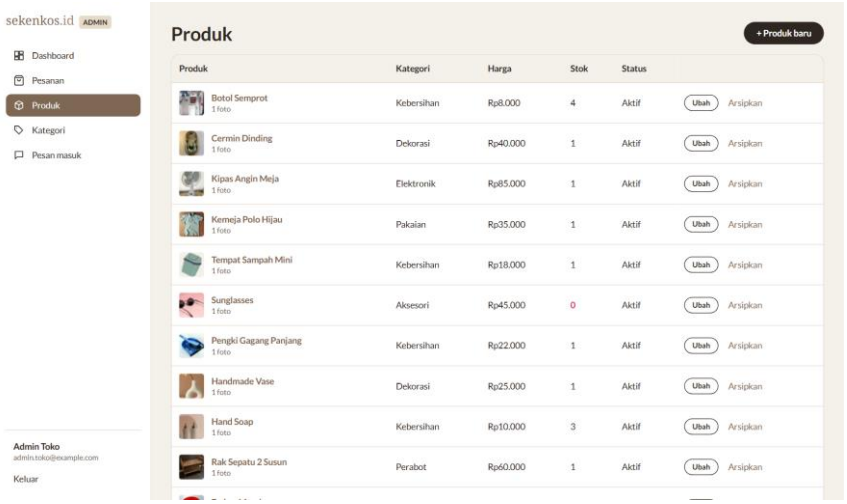
- Login menggunakan akun admin. Anda akan diarahkan langsung ke halaman **Dashboard**.
- Dashboard menampilkan **Pendapatan 30 hari**, **Pesanan 30 hari**, **Produk aktif**, bagian **Perlu tindakan**, grafik **Pendapatan 7 hari terakhir**, **Stok habis**, dan **Pesanan yang menunggu**.
- Gunakan menu di sebelah kiri: **Dashboard**, **Pesanan**, **Produk**, **Kategori**, dan **Pesan masuk**. Klik **Keluar** di bagian bawah untuk keluar dari akun.



Gambar 6.1 Dashboard admin

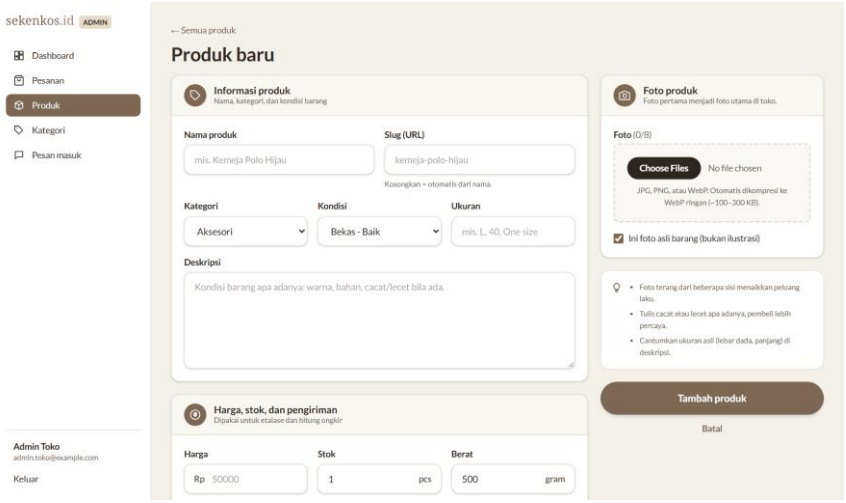
2. Kelola Produk

- Klik menu **Produk** untuk melihat semua produk beserta **Kategori**, **Harga**, **Stok**, dan **Status**.
- Klik **Ubah** untuk mengedit produk, atau **Arsipkan** untuk menyembunyikan produk dari toko. Produk yang diarsipkan dapat ditampilkan lagi dengan **Aktifkan**.



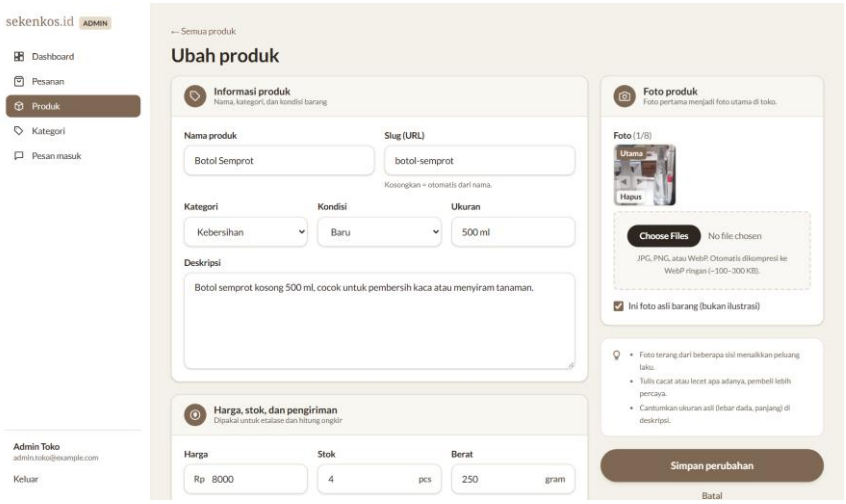
Gambar 6.2 Daftar produk di halaman admin

3. **Tambah Produk Baru**
   - Klik tombol **Produk baru** di kanan atas halaman **Produk**.
   - Pada **Informasi produk**, isi **Nama produk**, pilih **Kategori** dan **Kondisi**, isi **Ukuran** dan **Deskripsi**. Kolom **Slug (URL)** boleh dikosongkan.
   - Pada **Harga, stok, dan pengiriman**, isi **Harga**, **Stok**, dan **Berat** dalam gram, lalu pilih **Status**: **Aktif** atau **Diarsipkan**.
   - Pada **Foto produk**, klik **Choose File** atau **Pilih File** untuk mengunggah foto (maksimal 8). Foto pertama menjadi foto utama.
   - Klik **Tambah produk** untuk menyimpan.



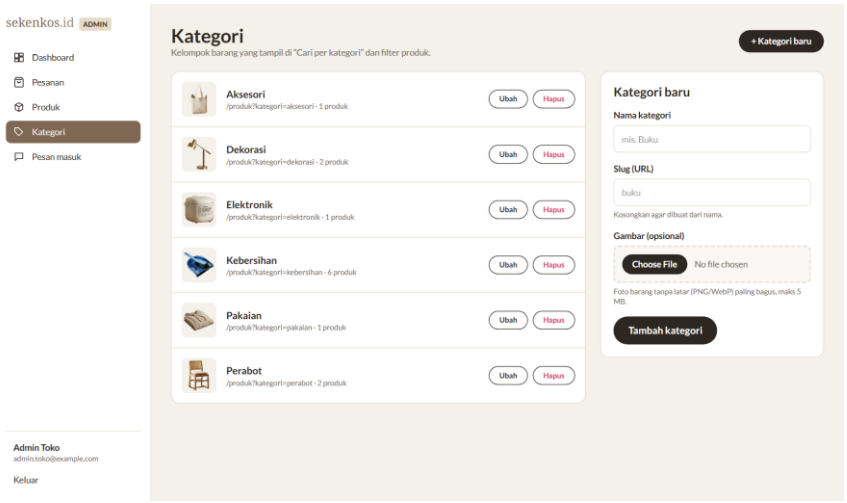
Gambar 6.3 Formulir Produk baru

4. **Ubah Produk**
   - Klik **Ubah** pada produk yang ingin diedit. Halaman **Ubah produk** akan tampil.
   - Atur urutan foto dengan tombol ◀ dan ▶, jadikan foto utama dengan **Utama**, atau hapus foto dengan **Hapus**.
   - Klik **Simpan perubahan** setelah selesai.



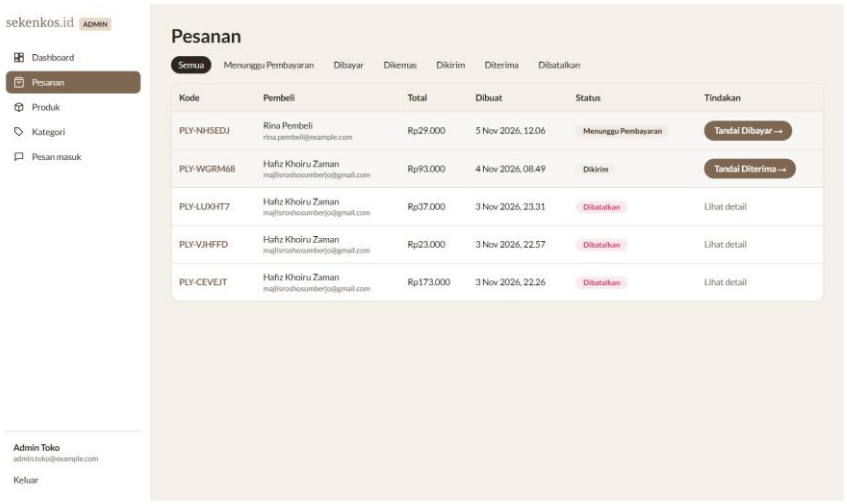
Gambar 6.4 Halaman Ubah produk

5. **Kelola Kategori**
   - Klik menu **Kategori**. Untuk menambah kategori, isi **Nama kategori** dan gambar kategori pada formulir **Kategori baru**, lalu klik **Tambah kategori**.
   - Klik **Ubah** untuk mengedit kategori, atau **Hapus** lalu **Ya, hapus** untuk menghapusnya.



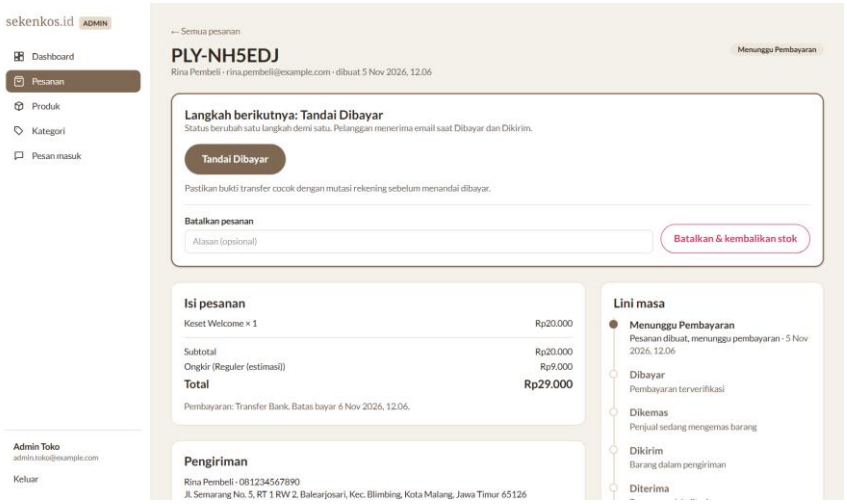
Gambar 6.5 Halaman Kategori

6. **Memproses Pesanan**
   - Klik menu **Pesanan**. Gunakan tombol filter di atas tabel untuk menampilkan pesanan berdasarkan status.
   - Pesanan yang perlu diproses diberi latar berwarna dan tombol langkah berikutnya pada kolom **Tindakan**. Klik tombol tersebut untuk membuka detail pesanan.
   - Status pesanan diubah satu langkah demi satu langkah melalui kotak **Langkah berikutnya** di bagian atas halaman detail.



Gambar 6.6 Daftar pesanan di halaman admin

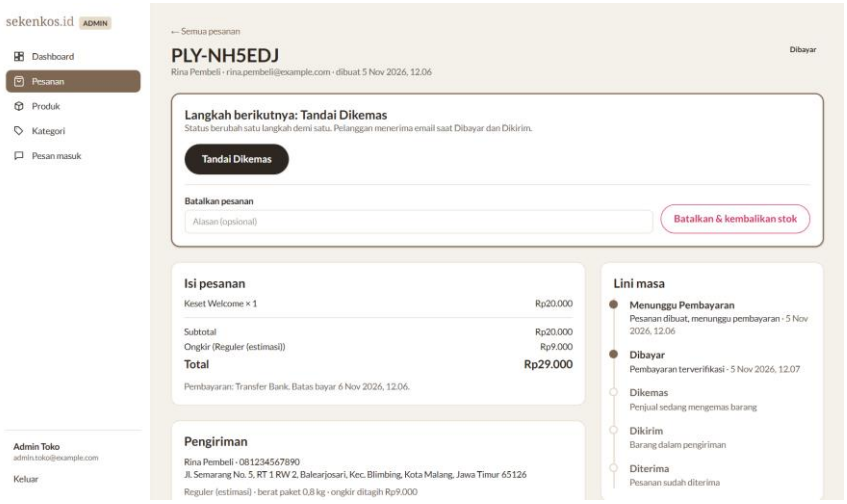
7. **Tandai Dibayar**
   - Cocokkan bukti transfer dari pembeli dengan mutasi rekening toko.
   - Jika sudah sesuai, klik **Tandai Dibayar**. Pembeli menerima email bahwa pembayaran sudah diterima.



Gambar 6.7 Langkah Tandai Dibayar

8. Tandai Dikemas

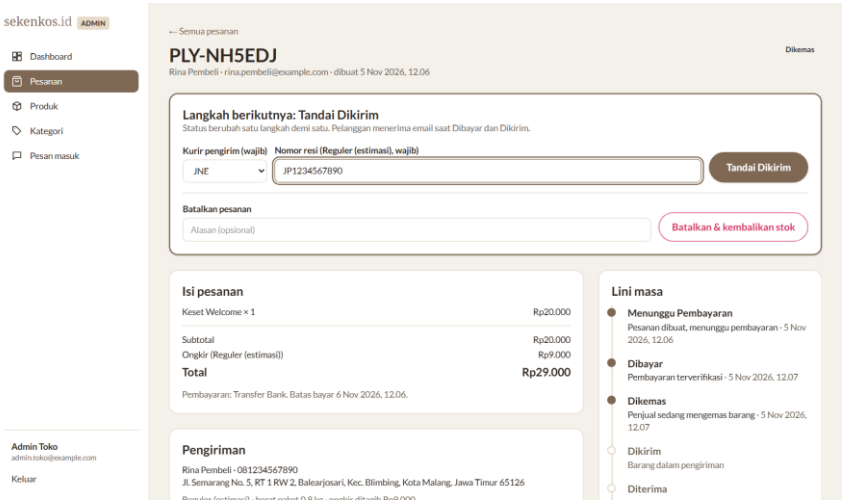
- Setelah barang selesai dikemas, klik **Tandai Dikemas**.



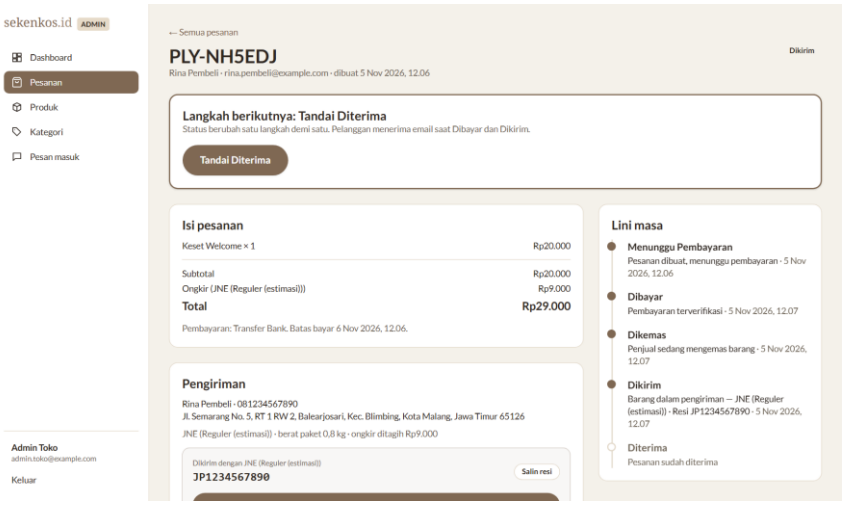
Gambar 6.8 Langkah Tandai Dikemas

9. Tandai Dikirim

- Pilih **Kurir pengirim** jika kolom tersebut muncul, lalu isi **Nomor resi** dari kurir.
- Klik **Tandai Dikirim**. Pembeli menerima email berisi nomor resi dan tautan pelacakan kurir.



Gambar 6.9 Mengisi nomor resi



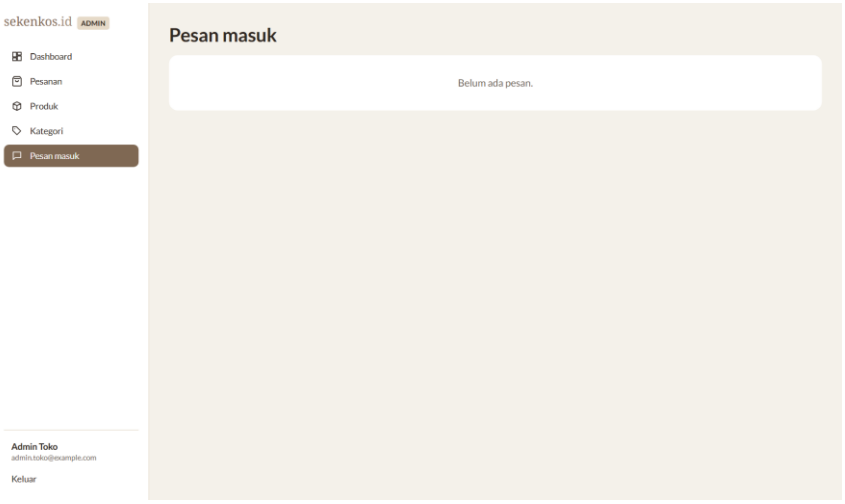
Gambar 6.10 Pesanan setelah ditandai dikirim

10. Tandai Diterima atau Batalkan Pesanan

- Klik **Tandai Diterima** jika paket sudah sampai tetapi pembeli belum mengonfirmasi.
- Untuk membatalkan pesanan yang belum dikirim, isi **Alasan (opsional)** lalu klik **Batalkan & kembalikan stok**. Stok barang akan dikembalikan.
- Pesanan yang tidak dibayar dalam 24 jam dibatalkan otomatis oleh sistem.

11. Pesan Masuk

- Klik menu **Pesan masuk** untuk membaca pesan dari formulir **Hubungi kami**.
- Klik **Tandai selesai** setelah pesan ditanggapi, atau **Tandai belum selesai** untuk membukanya kembali.



Gambar 6.11 Halaman Pesan masuk

Source Code Website

Berikut potongan source code inti website sekenkos.id. File konfigurasi rahasia (.env) dan kredensial tidak disertakan.

prisma/schema.prisma

```
// Skema database sekenkos.id (MVP). Harga dalam rupiah penuh (Int), bukan sen.
// Tabel sessions/accounts/verifications dipakai Better Auth (autentikasi).

generator client {
  provider = "prisma-client"
  output   = "../src/generated/prisma"
}

datasource db {
  provider = "postgresql"
}

enum Role {
  BUYER
  ADMIN
}

enum ProductCondition {
  BARU
  SEPERTI_BARU
  BEKAS_BAIK
}

enum ProductStatus {
  ACTIVE
  ARCHIVED
}

enum OrderStatus {
  PENDING_PAYMENT
  PAID
  PACKED
  SHIPPED
  RECEIVED
  CANCELLED
}

enum PaymentMethod {
  BANK_TRANSFER
  EWALLET
  // QRIS dinamis (nominal terkunci), dikonfirmasi manual lewat WhatsApp seperti transfer.
  QRIS
  // Bayar online lewat payment gateway (Midtrans): VA bank, QRIS, e-wallet, kartu. Status dibayar
otomatis via webhook.
  // Dinonaktifkan di checkout; nilai enum dipertahankan agar pesanan lama tetap valid.
  GATEWAY
}

model User {
  id            String   @id @default(cuid())
  name          String
  email         String   @unique
  emailVerified Boolean  @default(false)
  image         String?
  role          Role     @default(BUYER)
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  sessions  Session[]
  accounts  Account[]
```

```
  cartItems CartItem[]
  addresses Address[]
  orders    Order[]
  events    OrderEvent[]

  @@map("users")
}

model Address {
  id          String   @id @default(cuid())
  userId      String
  recipient   String
  phone       String
  street      String
  // Alamat terstruktur: city = kabupaten/kota. villageCode = kode desa/kelurahan (Kepmendagri),
dipakai mengisi ulang form.
  // Baris lama (sebelum alamat terstruktur) punya province/district/village/villageCode kosong.
  province    String   @default("")
  city        String
  district    String   @default("")
  village     String   @default("")
  villageCode String   @default("")
  zip         String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@map("addresses")
}

model Category {
  id   String @id @default(cuid())
  slug String @unique
  name String
  icon String

  products Product[]

  @@map("categories")
}

model Product {
  id          String           @id @default(cuid())
  slug        String           @unique
  name        String
  categoryId  String
  price       Int
  condition   ProductCondition
  size        String
  // Berat barang (gram) untuk menghitung ongkir. Diisi admin; bawaan 500 g.
  weightGrams Int              @default(500)
  description String
  // Barang preloved umumnya 1 unit. Stok dikurangi atomik saat pesanan dibuat
  // dan dikembalikan saat pesanan batal/kedaluwarsa.
  stock       Int
  images      String[]
  // true = foto produk asli; false = ilustrasi (dipakai untuk gaya tampilan).
  isPhoto     Boolean          @default(false)
  popularity  Int              @default(0)
  status      ProductStatus    @default(ACTIVE)
  createdAt   DateTime         @default(now())
  updatedAt   DateTime         @updatedAt

  category   Category    @relation(fields: [categoryId], references: [id])
  orderItems OrderItem[]
  cartItems  CartItem[]

  @@index([categoryId, status])
```

```
  @@index([status, createdAt])
  @@map("products")
}

model Promo {
  id                String    @id @default(cuid())
  code              String    @unique
  percent           Int
  validUntil        DateTime?
  quota             Int?
  usedCount         Int       @default(0)
  firstPurchaseOnly Boolean   @default(false)
  active            Boolean   @default(true)
  createdAt         DateTime  @default(now())

  orders Order[]

  @@map("promos")
}

model Order {
  id               String        @id @default(cuid())
  // Kode yang tampil ke pembeli dan dipakai di pesan WhatsApp, mis. PLY-7K3M9Q.
  code             String        @unique
  userId           String
  status           OrderStatus   @default(PENDING_PAYMENT)
  subtotal         Int
  discount         Int           @default(0)
  shippingCost     Int
  total            Int
  courier          String
  // Berat paket (gram) yang dipakai saat menghitung ongkir, termasuk bahan kemasan.
  totalWeight      Int           @default(0)
  trackingNumber   String?
  paymentMethod    PaymentMethod
  // Diisi untuk pembayaran GATEWAY: id & tautan bayar Midtrans (id = kode pesanan), dan kanal yang
dipakai pembeli (mis. BCA, QRIS).
  gatewayInvoiceId String?       @unique
  gatewayUrl       String?
  paymentChannel   String?
  promoId          String?
  // Salinan alamat saat checkout agar riwayat tidak berubah bila alamat diedit.
  shipName         String
  shipPhone        String
  shipStreet       String
  shipVillage      String        @default("")
  shipDistrict     String        @default("")
  shipCity         String
  shipProvince     String        @default("")
  shipZip          String
  expiresAt        DateTime
  paidAt           DateTime?
  shippedAt        DateTime?
  receivedAt       DateTime?
  cancelledAt      DateTime?
  createdAt        DateTime      @default(now())
  updatedAt        DateTime      @updatedAt

  user   User         @relation(fields: [userId], references: [id])
  promo  Promo?       @relation(fields: [promoId], references: [id])
  items  OrderItem[]
  events OrderEvent[]

  @@index([userId, createdAt])
  @@index([status, expiresAt])
  @@index([trackingNumber])
  @@map("orders")
}

model OrderItem {
  id            String @id @default(cuid())
```

```
  orderId       String
  productId     String
  // Nama dan harga disalin saat checkout.
  nameSnapshot  String
  priceSnapshot Int
  qty           Int

  order   Order   @relation(fields: [orderId], references: [id], onDelete: Cascade)
  product Product @relation(fields: [productId], references: [id])

  @@index([orderId])
  @@index([productId])
  @@map("order_items")
}

// Jejak perubahan status: siapa (admin/sistem) mengubah apa, dan kapan.
model OrderEvent {
  id        String      @id @default(cuid())
  orderId   String
  status    OrderStatus
  actorId   String?
  note      String?
  createdAt DateTime    @default(now())

  order Order @relation(fields: [orderId], references: [id], onDelete: Cascade)
  actor User? @relation(fields: [actorId], references: [id])

  @@index([orderId, createdAt])
  @@map("order_events")
}

model Message {
  id        String   @id @default(cuid())
  name      String
  email     String
  body      String
  handled   Boolean  @default(false)
  createdAt DateTime @default(now())

  @@map("messages")
}

model Session {
  id        String   @id @default(cuid())
  expiresAt DateTime
  token     String   @unique
  ipAddress String?
  userAgent String?
  userId    String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@map("sessions")
}

// Kredensial login. Password disimpan sebagai hash oleh Better Auth, bukan teks asli.
model Account {
  id                    String    @id @default(cuid())
  accountId             String
  providerId            String
  userId                String
  accessToken           String?
  refreshToken          String?
  idToken               String?
  accessTokenExpiresAt  DateTime?
  refreshTokenExpiresAt DateTime?
```

```
  scope                 String?
  password              String?
  createdAt             DateTime  @default(now())
  updatedAt             DateTime  @updatedAt

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@map("accounts")
}

model Verification {
  id         String   @id @default(cuid())
  identifier String
  value      String
  expiresAt  DateTime
  createdAt  DateTime @default(now())
  updatedAt  DateTime @default(now()) @updatedAt

  @@index([identifier])
  @@map("verifications")
}

// Keranjang milik akun yang sedang login. Pengunjung tanpa akun memakai cookie (lihat src/lib/cart-
server.ts).
// Saat login, isi cookie digabung ke sini lalu cookie dikosongkan.
model CartItem {
  id        String   @id @default(cuid())
  userId    String
  productId String
  qty       Int
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  user    User    @relation(fields: [userId], references: [id], onDelete: Cascade)
  product Product @relation(fields: [productId], references: [id], onDelete: Cascade)

  @@unique([userId, productId])
  @@index([userId])
  @@map("cart_items")
}

// Foto produk yang diunggah admin saat Supabase Storage belum dikonfigurasi. Disajikan lewat
/img/[id].
model ProductImage {
  id          String   @id @default(cuid())
  data        Bytes
  contentType String
  size        Int
  createdAt   DateTime @default(now())

  @@map("product_images")
}
```

src/lib/auth.ts

```
import "server-only";
import { betterAuth } from "better-auth";
import { after } from "next/server";
import { prismaAdapter } from "better-auth/adapters/prisma";
import { nextCookies } from "better-auth/next-js";
import { db } from "@/lib/db";
import { sendEmail } from "@/lib/email";
import { SITE } from "@/lib/site";

// Batas percobaan per menit per IP. Bisa dinaikkan hanya untuk tes otomatis (lihat
playwright.config.ts).
const strictMax = Number(process.env.AUTH_RATE_LIMIT_MAX) || 5;
```

```
// Jeda minimal antar email reset untuk akun yang sama.
const RESET_COOLDOWN_SEC = 120;

// Asal (origin) yang boleh memanggil API auth selain baseURL. Di Vercel, domain produksi dan URL
deploy/preview
// diisi otomatis oleh Vercel, jadi login tidak ditolak "Invalid origin" saat dibuka lewat domain
tersebut.
const trustedOrigins = [
  process.env.NEXT_PUBLIC_SITE_URL,
  ...[process.env.VERCEL_PROJECT_PRODUCTION_URL, process.env.VERCEL_URL,
process.env.VERCEL_BRANCH_URL].map((h) => (h ? `https://${h}` : undefined)),
].filter((o): o is string => Boolean(o));

export const auth = betterAuth({
  baseURL: process.env.BETTER_AUTH_URL ?? SITE.url,
  trustedOrigins,
  secret: process.env.BETTER_AUTH_SECRET,
  database: prismaAdapter(db, { provider: "postgresql" }),
  emailAndPassword: {
    enabled: true,
    minPasswordLength: 8,
    maxPasswordLength: 128,
    // Tautan reset berlaku 1 jam dan hanya sekali pakai (F-02); sesi lama dicabut setelah reset.
    resetPasswordTokenExpiresIn: 60 * 60,
    revokeSessionsOnPasswordReset: true,
    sendResetPassword: async ({ user, url }) => {
      // Batas per akun (bukan per IP): maksimal 1 email reset per RESET_COOLDOWN_SEC. Token yang baru
dibuat
      // sudah tercatat saat fungsi ini dipanggil, jadi lebih dari 1 token baru = permintaan beruntun,
email dilewati.
      // Respons ke pengguna tetap sama, jadi tidak membocorkan apakah email terdaftar.
      const recent = await db.verification.count({
        where: {
          value: user.id,
          identifier: { startsWith: "reset-password:" },
          createdAt: { gt: new Date(Date.now() - RESET_COOLDOWN_SEC * 1000) },
        },
      });
      if (recent > 1) return;

      const send = () =>
        sendEmail({
          to: user.email,
          subject: "Reset password sekenkos.id",
          text: `Halo ${user.name},\n\nKlik tautan berikut untuk membuat password baru (berlaku 1
jam):\n${url}\n\nJika kamu tidak meminta ini, abaikan email ini.`,
        }).catch((e) => console.error("Gagal mengirim email reset password:", e));
      // after(): dijalankan setelah respons terkirim (waktu respons tidak membocorkan keberadaan akun)
dan
      // di serverless tetap diselesaikan platform. Di luar lingkup request, jatuh ke pengiriman biasa.
      try {
        after(send);
      } catch {
        void send();
      }
    },
  },
  user: {
    additionalFields: {
      // input:false = tidak bisa diisi dari form daftar, sehingga tak seorang pun bisa mendaftar
sebagai admin.
      role: { type: "string", defaultValue: "BUYER", input: false },
    },
  },
  session: { expiresIn: 60 * 60 * 24 * 30 },
  // Pembatasan percobaan (NFR keamanan). Penyimpanan di memori per instance: cukup untuk MVP,
  // pindah ke database/Redis bila trafik naik.
  rateLimit: {
    enabled: true,
    window: 60,
    max: 100,
```

```
    customRules: {
      "/sign-in/email": { window: 60, max: strictMax },
      "/sign-up/email": { window: 60, max: strictMax },
      "/request-password-reset": { window: 60, max: 3 },
    },
  },
  plugins: [nextCookies()],
});
```

src/lib/orders.ts

```
import "server-only";
import type { Prisma } from "@/generated/prisma/client";
import type { OrderStatus, PaymentMethod } from "@/generated/prisma/enums";
import type { CartLine } from "@/lib/cart";
import { db } from "@/lib/db";
import { generateOrderCode } from "@/lib/order-code";
import { CANCELLABLE, PREVIOUS } from "@/lib/order-status";
import { calcTotals } from "@/lib/pricing";
import { expireInvoice } from "@/lib/gateway";

export const ORDER_TTL_MS = 24 * 60 * 60 * 1000;

/** Kesalahan yang aman ditampilkan ke pembeli (pesannya berbahasa Indonesia). */
export class OrderError extends Error {}

type Tx = Prisma.TransactionClient;

export type Address = {
  recipient: string;
  phone: string;
  street: string;
  village: string;
  district: string;
  /** Kabupaten/kota. */
  city: string;
  province: string;
  zip: string;
};

export type CheckoutInput = {
  userId: string;
  lines: CartLine[];
  address: Address;
  /** Pilihan ongkir yang sudah dihitung ulang di server (lihat resolveShipping). */
  shipping: { name: string; cost: number };
  /** Berat paket (gram) yang dipakai untuk tarif tersebut. */
  totalWeight: number;
  paymentMethod: PaymentMethod;
  promoCode?: string;
};

/** Memeriksa kode promo tanpa mengubah apa pun. Dipakai untuk pratinjau dan di dalam transaksi
checkout. */
export async function validatePromo(client: Tx | typeof db, rawCode: string, userId: string) {
  const code = rawCode.trim().toUpperCase();
  const promo = await client.promo.findUnique({ where: { code } });
  if (!promo || !promo.active) throw new OrderError("Kode promo tidak valid.");
  if (promo.validUntil && promo.validUntil < new Date()) throw new OrderError("Kode promo sudah
kedaluwarsa.");
  if (promo.quota !== null && promo.usedCount >= promo.quota) throw new OrderError("Kuota kode promo
sudah habis.");
  if (promo.firstPurchaseOnly) {
    const previous = await client.order.count({ where: { userId, status: { not: "CANCELLED" } } });
    if (previous > 0) throw new OrderError("Kode ini hanya berlaku untuk pembelian pertama.");
  }
  return promo;
}
```

```
const isCodeCollision = (e: unknown) =>
  typeof e === "object" && e !== null && (e as { code?: string }).code === "P2002";

export async function createOrder(input: CheckoutInput) {
  if (input.lines.length === 0) throw new OrderError("Keranjang kosong.");

  for (let attempt = 0; attempt < 5; attempt++) {
    try {
      return await db.$transaction((tx) => build(tx, input, generateOrderCode()));
    } catch (e) {
      if (isCodeCollision(e)) continue; // kode pesanan kebetulan sama; buat ulang
      throw e;
    }
  }
  throw new OrderError("Gagal membuat pesanan. Silakan coba lagi.");
}

async function build(tx: Tx, input: CheckoutInput, code: string) {
  const products = await tx.product.findMany({
    where: { id: { in: input.lines.map((l) => l.id) }, status: "ACTIVE" },
  });
  const byId = new Map(products.map((p) => [p.id, p]));

  let subtotal = 0;
  const items: { productId: string; nameSnapshot: string; priceSnapshot: number; qty: number }[] = [];
  for (const line of input.lines) {
    const p = byId.get(line.id);
    if (!p) throw new OrderError("Salah satu barang di keranjang sudah tidak dijual.");
    // Pengurangan stok bersyarat dalam satu perintah: bila dua pembeli berebut 1 unit, hanya satu yang
    // menghasilkan count 1. Pembeli lain mendapat 0 dan seluruh transaksi dibatalkan (rollback).
    const taken = await tx.product.updateMany({
      where: { id: p.id, status: "ACTIVE", stock: { gte: line.qty } },
      data: { stock: { decrement: line.qty } },
    });
    if (taken.count === 0) throw new OrderError(`Maaf, stok ${p.name} sudah tidak cukup.`);
    subtotal += p.price * line.qty;
    items.push({ productId: p.id, nameSnapshot: p.name, priceSnapshot: p.price, qty: line.qty });
  }

  let promoId: string | undefined;
  let percent = 0;
  if (input.promoCode?.trim()) {
    const promo = await validatePromo(tx, input.promoCode, input.userId);
    // Penghitung dinaikkan hanya bila nilainya belum berubah sejak dibaca (cegah kuota terlampaui).
    const claimed = await tx.promo.updateMany({
      where: { id: promo.id, usedCount: promo.usedCount },
      data: { usedCount: { increment: 1 } },
    });
    if (claimed.count === 0) throw new OrderError("Kode promo baru saja habis dipakai. Silakan coba
lagi.");
    promoId = promo.id;
    percent = promo.percent;
  }

  const totals = calcTotals(subtotal, input.shipping.cost, percent);
  const { address: a } = input;
  return tx.order.create({
    data: {
      code,
      userId: input.userId,
      subtotal: totals.subtotal,
      discount: totals.discount,
      shippingCost: totals.shipping,
      total: totals.total,
      courier: input.shipping.name,
      totalWeight: input.totalWeight,
      paymentMethod: input.paymentMethod,
      promoId,
      shipName: a.recipient,
      shipPhone: a.phone,
      shipStreet: a.street,
```

```
      shipVillage: a.village,
      shipDistrict: a.district,
      shipCity: a.city,
      shipProvince: a.province,
      shipZip: a.zip,
      expiresAt: new Date(Date.now() + ORDER_TTL_MS),
      items: { create: items },
      events: { create: { status: "PENDING_PAYMENT", actorId: input.userId, note: "Pesanan dibuat" } },
    },
    include: { items: true, user: { select: { email: true, name: true } } },
  });
}

/**
 * Memajukan status satu langkah. Idempoten: bila status saat ini bukan status asal yang sah
 * (mis. sudah ditandai dibayar oleh admin lain), tidak ada yang berubah dan `changed` bernilai false.
 */
export async function advanceOrder(
  orderId: string,
  to: Exclude<OrderStatus, "PENDING_PAYMENT" | "CANCELLED">,
  actorId: string | null,
  opts: { trackingNumber?: string; note?: string } = {},
) {
  const from = PREVIOUS[to]!;
  if (to === "SHIPPED" && !opts.trackingNumber?.trim()) throw new OrderError("Nomor resi wajib
diisi.");
  const now = new Date();

  return db.$transaction(async (tx) => {
    const moved = await tx.order.updateMany({
      where: { id: orderId, status: from },
      data: {
        status: to,
        ...(to === "PAID" && { paidAt: now }),
        ...(to === "SHIPPED" && { shippedAt: now, trackingNumber: opts.trackingNumber!.trim() }),
        ...(to === "RECEIVED" && { receivedAt: now }),
      },
    });
    const order = await tx.order.findUnique({
      where: { id: orderId },
      include: { user: { select: { email: true, name: true } } },
    });
    if (moved.count === 0) return { changed: false as const, order };
    await tx.orderEvent.create({ data: { orderId, status: to, actorId, note: opts.note } });
    return { changed: true as const, order };
  });
}

/** Membatalkan pesanan, mengembalikan stok dan kuota promo. Idempoten seperti advanceOrder. */
export async function cancelOrder(orderId: string, actorId: string | null, note: string) {
  return db.$transaction(async (tx) => {
    const cancelled = await tx.order.updateMany({
      where: { id: orderId, status: { in: CANCELLABLE } },
      data: { status: "CANCELLED", cancelledAt: new Date() },
    });
    const order = await tx.order.findUnique({
      where: { id: orderId },
      include: { items: true, user: { select: { email: true, name: true } } },
    });
    if (cancelled.count === 0 || !order) return { changed: false as const, order };

    for (const item of order.items) {
      await tx.product.update({ where: { id: item.productId }, data: { stock: { increment: item.qty } }
});
    }
    if (order.promoId) {
      await tx.promo.updateMany({ where: { id: order.promoId, usedCount: { gt: 0 } }, data: {
usedCount: { decrement: 1 } } });
    }
    await tx.orderEvent.create({ data: { orderId, status: "CANCELLED", actorId, note } });
    return { changed: true as const, order };
  }).then(async (r) => {
```

```
    // Pembatalan sebelum invoice kedaluwarsa: tutup invoice agar tidak bisa dibayar lagi.
    if (r.changed && r.order?.gatewayInvoiceId && r.order.expiresAt > new Date()) await
expireInvoice(r.order.gatewayInvoiceId);
    return r;
  });
}

/** Membatalkan pesanan yang melewati batas bayar dan mengembalikan stoknya (F-14). */
export async function expireOverdueOrders(limit = 100) {
  const overdue = await db.order.findMany({
    where: { status: "PENDING_PAYMENT", expiresAt: { lt: new Date() } },
    select: { id: true },
    orderBy: { expiresAt: "asc" },
    take: limit,
  });
  const cancelled = [];
  for (const { id } of overdue) {
    const r = await cancelOrder(id, null, "Kedaluwarsa: tidak dibayar dalam 24 jam");
    if (r.changed) cancelled.push(r.order!);
  }
  return cancelled;
}
```

src/lib/order-status.ts

```
import type { OrderStatus } from "@/generated/prisma/enums";

export const STATUS_LABEL: Record<OrderStatus, string> = {
  PENDING_PAYMENT: "Menunggu Pembayaran",
  PAID: "Dibayar",
  PACKED: "Dikemas",
  SHIPPED: "Dikirim",
  RECEIVED: "Diterima",
  CANCELLED: "Dibatalkan",
};

/** Urutan tahap normal (tanpa Dibatalkan) untuk ditampilkan sebagai lini masa. */
export const FLOW: OrderStatus[] = ["PENDING_PAYMENT", "PAID", "PACKED", "SHIPPED", "RECEIVED"];

/** Status asal yang sah untuk setiap status tujuan (alur maju satu langkah). */
export const PREVIOUS: Partial<Record<OrderStatus, OrderStatus>> = {
  PAID: "PENDING_PAYMENT",
  PACKED: "PAID",
  SHIPPED: "PACKED",
  RECEIVED: "SHIPPED",
};

/** Pesanan baru boleh dibatalkan sebelum dikirim; setelah dikirim tidak lagi. */
export const CANCELLABLE: OrderStatus[] = ["PENDING_PAYMENT", "PAID", "PACKED"];

export const canAdvance = (from: OrderStatus, to: OrderStatus) => PREVIOUS[to] === from;
export const canCancel = (from: OrderStatus) => CANCELLABLE.includes(from);

/** Langkah berikutnya yang ditawarkan ke admin pada sebuah pesanan. */
export function nextStep(status: OrderStatus): { to: OrderStatus; label: string } | null {
  switch (status) {
    case "PENDING_PAYMENT":
      return { to: "PAID", label: "Tandai Dibayar" };
    case "PAID":
      return { to: "PACKED", label: "Tandai Dikemas" };
    case "PACKED":
      return { to: "SHIPPED", label: "Tandai Dikirim" };
    case "SHIPPED":
      return { to: "RECEIVED", label: "Tandai Diterima" };
    default:
      return null;
  }
}
```

src/lib/couriers.ts

```
/**
 * Kurir yang dipakai toko dan halaman pelacakan resminya (gratis, dibuka di situs kurir).
 * Situs kurir tidak menerima resi lewat URL secara resmi, jadi pelanggan menyalin resi lalu
menempelkannya di sana.
 */
export const CARRIERS = [
  { code: "jne", name: "JNE", trackUrl: "https://www.jne.co.id/tracking-package", match: /\bjne\b/i },
  { code: "jnt", name: "J&T Express", trackUrl: "https://jet.co.id/track", match: /j\s*&\s*t|\bjnt\b/i
},
  { code: "sicepat", name: "SiCepat", trackUrl: "https://www.sicepat.com/checkAwb", match: /sicepat/i
},
  { code: "anteraja", name: "AnterAja", trackUrl: "https://anteraja.id/tracking", match: /anteraja/i },
] as const;

export type Carrier = (typeof CARRIERS)[number];

/** Mengenali kurir dari nama layanan yang tersimpan di pesanan, mis. "JNE Reguler" atau "J&T EZ". */
export function detectCarrier(courierName: string | null | undefined): Carrier | null {
  if (!courierName) return null;
  return CARRIERS.find((c) => c.match.test(courierName)) ?? null;
}

export const carrierByCode = (code: string) => CARRIERS.find((c) => c.code === code) ?? null;
```

src/app/(shop)/checkout/actions.ts

```
"use server";

import { refresh } from "next/cache";
import { after } from "next/server";
import { redirect } from "next/navigation";
import { z } from "zod";
import type { CartLine } from "@/lib/cart";
import { writeCart, readCart } from "@/lib/cart-server";
import { db } from "@/lib/db";
import { formatRupiah } from "@/lib/format";
// import { GatewayError, createInvoice, gatewayEnabled } from "@/lib/gateway";
import { runExpiry } from "@/lib/maintenance";
import { emailOrderCreated } from "@/lib/order-emails";
import { OrderError, /* cancelOrder, */ createOrder, validatePromo } from "@/lib/orders";
import { requireUser } from "@/lib/session";
import { type ShippingOption } from "@/lib/shipping";
import { packageWeight } from "@/lib/shipping-calc";
import { ShippingError, quoteShipping, resolveShipping, shippingSettings } from "@/lib/shipping-quote";
// import { SITE } from "@/lib/site";
import { resolveVillage } from "@/lib/wilayah";

export type CheckoutState = { error?: string; refreshShipping?: boolean };
export type ShippingState = { options: ShippingOption[]; error?: undefined } | { options?: undefined;
error: string };
export type PromoState = { ok: boolean; message: string; percent?: number; code?: string };

const phone = z
  .string()
  .transform((v) => v.replace(/[\s-]/g, ""))
  .pipe(z.string().regex(/^(0|\+62)\d{8,13}$/, "Nomor HP tidak valid."));

const schema = z.object({
  recipient: z.string().trim().min(2, "Nama penerima wajib diisi.").max(80),
  phone,
  street: z.string().trim().min(5, "Detail alamat (jalan, nomor, RT/RW) wajib diisi.").max(200),
  villageCode: z.string().trim().min(1, "Pilih provinsi sampai desa / kelurahan."),
  zip: z.string().trim().regex(/^\d{5}$/, "Kode pos harus 5 digit."),
  courier: z.string().trim().min(1, "Pilih jasa pengiriman.").max(60),
  // Ongkir yang tampil di layar saat pembeli memilih; dibandingkan dengan hitungan ulang di server.
  shippingCost: z.coerce.number().int().min(0),
```

```
  // "GATEWAY" (Midtrans) dinonaktifkan; tambahkan kembali bersama blok yang dikomentari di bawah bila
dibutuhkan.
  payment: z.enum(["BANK_TRANSFER", "EWALLET", "QRIS"], "Pilih metode pembayaran."),
  promo: z.string().trim().max(40).optional(),
});

const text = (v: FormDataEntryValue | null) => (typeof v === "string" ? v : "");

export async function checkPromo(code: string): Promise<PromoState> {
  const user = await requireUser("/checkout");
  const trimmed = code.trim();
  if (!trimmed) return { ok: false, message: "Masukkan kode promo." };
  try {
    const promo = await validatePromo(db, trimmed, user.id);
    return { ok: true, message: `Diskon ${promo.percent}% dipakai.`, percent: promo.percent, code:
promo.code };
  } catch (e) {
    if (e instanceof OrderError) return { ok: false, message: e.message };
    throw e;
  }
}

/** Berat paket dan nilai barang untuk isi keranjang saat ini, dihitung dari data di database (bukan
dari klien). */
async function cartParcel(lines: CartLine[]) {
  const products = await db.product.findMany({
    where: { id: { in: lines.map((l) => l.id) }, status: "ACTIVE" },
    select: { id: true, price: true, weightGrams: true },
  });
  const byId = new Map(products.map((p) => [p.id, p]));
  const found = lines.flatMap((l) => {
    const p = byId.get(l.id);
    return p ? [{ weightGrams: p.weightGrams, price: p.price, qty: l.qty }] : [];
  });
  return {
    weight: packageWeight(found, shippingSettings().packingGrams),
    value: found.reduce((sum, p) => sum + p.price * p.qty, 0),
  };
}

/** Pilihan ongkir untuk kode pos tujuan, berdasarkan berat isi keranjang. Dipanggil dari form
checkout. */
export async function getShippingOptions(zip: string, villageCode = ""): Promise<ShippingState> {
  await requireUser("/checkout");
  const lines = await readCart();
  if (lines.length === 0) return { error: "Keranjangmu kosong." };
  try {
    const parcel = await cartParcel(lines);
    const place = await resolveVillage(villageCode);
    return { options: await quoteShipping(zip, parcel.weight, parcel.value, place?.regencyCode) };
  } catch (e) {
    if (e instanceof ShippingError) return { error: e.message };
    throw e;
  }
}

export async function placeOrder(_prev: CheckoutState, formData: FormData): Promise<CheckoutState> {
  const user = await requireUser("/checkout");

  const parsed = schema.safeParse({
    recipient: text(formData.get("recipient")),
    phone: text(formData.get("phone")),
    street: text(formData.get("street")),
    villageCode: text(formData.get("villageCode")),
    zip: text(formData.get("zip")),
    courier: text(formData.get("courier")),
    shippingCost: text(formData.get("shippingCost")),
    payment: text(formData.get("payment")),
    promo: text(formData.get("promo")) || undefined,
  });
  if (!parsed.success) return { error: parsed.error.issues[0].message };
```

```
  const d = parsed.data;

  // Nama wilayah diturunkan dari kode desa di server, bukan dipercaya dari klien.
  const place = await resolveVillage(d.villageCode);
  if (!place) return { error: "Alamat tidak valid. Pilih ulang provinsi sampai desa / kelurahan." };

  const lines = await readCart();
  if (lines.length === 0) return { error: "Keranjangmu kosong." };
  // if (d.payment === "GATEWAY" && !gatewayEnabled()) return { error: "Pembayaran online belum
tersedia. Pilih transfer manual." };

  // Ongkir selalu dihitung ulang di server dari berat barang di database dan tarif kurir terbaru.
  let shipping: ShippingOption;
  let totalWeight: number;
  try {
    const parcel = await cartParcel(lines);
    totalWeight = parcel.weight;
    shipping = await resolveShipping(d.zip, parcel.weight, d.courier, parcel.value, place.regencyCode);
  } catch (e) {
    if (e instanceof ShippingError) return { error: e.message, refreshShipping: true };
    throw e;
  }
  if (shipping.cost !== d.shippingCost) {
    return {
      error: `Tarif ongkir berubah menjadi ${formatRupiah(shipping.cost)}. Periksa totalnya lalu tekan
Buat Pesanan lagi.`,
      refreshShipping: true,
    };
  }

  // Lepaskan stok dari pesanan kedaluwarsa dulu agar tidak salah menolak pembeli.
  await runExpiry();

  let order;
  try {
    order = await createOrder({
      userId: user.id,
      lines,
      address: { recipient: d.recipient, phone: d.phone, street: d.street, village: place.village,
district: place.district, city: place.city, province: place.province, zip: d.zip },
      shipping: { name: shipping.name, cost: shipping.cost },
      totalWeight,
      paymentMethod: d.payment,
      promoCode: d.promo,
    });
  } catch (e) {
    if (e instanceof OrderError) return { error: e.message };
    throw e;
  }

  // Simpan alamat terakhir untuk checkout berikutnya.
  const addr = {
    recipient: d.recipient,
    phone: d.phone,
    street: d.street,
    province: place.province,
    city: place.city,
    district: place.district,
    village: place.village,
    villageCode: place.villageCode,
    zip: d.zip,
  };
  const existing = await db.address.findFirst({ where: { userId: user.id }, orderBy: { updatedAt:
"desc" } });
  if (existing) await db.address.update({ where: { id: existing.id }, data: addr });
  else await db.address.create({ data: { userId: user.id, ...addr } });

  const payUrl = `/pesanan/${order.code}`;
  // Payment gateway (Midtrans) dinonaktifkan. Untuk mengaktifkan lagi: ubah payUrl menjadi `let`, buka
komentar blok ini,
```

```
  // tambahkan "GATEWAY" ke enum `payment` dan CHECKOUT_PAYMENTS, serta impor GatewayError,
createInvoice, cancelOrder, SITE.
  // if (d.payment === "GATEWAY") {
  //   try {
  //     const invoice = await createInvoice({
  //       externalId: order.code,
  //       amount: order.total,
  //       description: `Pesanan ${order.code} - ${SITE.name}`,
  //       email: order.user.email,
  //       name: d.recipient,
  //       phone: d.phone,
  //       // Invoice berakhir sedikit sebelum pesanan kedaluwarsa agar tidak ada pembayaran setelah
stok dilepas.
  //       durationSeconds: Math.max(1800, Math.floor((order.expiresAt.getTime() - Date.now()) / 1000)
- 120),
  //       successUrl: `${SITE.url}/pesanan/${order.code}?bayar=ok`,
  //       failureUrl: `${SITE.url}/pesanan/${order.code}`,
  //     });
  //     await db.order.update({ where: { id: order.id }, data: { gatewayInvoiceId: invoice.id,
gatewayUrl: invoice.url } });
  //     payUrl = invoice.url;
  //   } catch (e) {
  //     // Tanpa invoice pesanan tidak bisa dibayar: batalkan dan kembalikan stok agar pembeli bisa
mencoba lagi.
  //     await cancelOrder(order.id, null, "Gagal membuat invoice pembayaran");
  //     if (e instanceof GatewayError) return { error: e.message };
  //     throw e;
  //   }
  // }

  await writeCart([]);
  after(() => emailOrderCreated(order, order.expiresAt));
  // Keranjang akun disimpan di database (bukan cookie), jadi header perlu direnderulang agar badge
keranjang kosong.
  refresh();
  redirect(payUrl);
}
```

src/app/admin/pesanan/actions.ts

```
"use server";

import { after } from "next/server";
import { redirect } from "next/navigation";
import { carrierByCode, detectCarrier } from "@/lib/couriers";
import { db } from "@/lib/db";
import { emailOrderCancelled, emailOrderPaid, emailOrderShipped } from "@/lib/order-emails";
import { OrderError, advanceOrder, cancelOrder } from "@/lib/orders";
import { requireAdmin } from "@/lib/session";

const text = (v: FormDataEntryValue | null) => (typeof v === "string" ? v : "");
const STEPS = ["PAID", "PACKED", "SHIPPED", "RECEIVED"] as const;
type Step = (typeof STEPS)[number];

async function findOrder(code: string) {
  const order = await db.order.findUnique({ where: { code }, select: { id: true } });
  if (!order) redirect("/admin/pesanan");
  return order;
}

export async function advance(formData: FormData) {
  const admin = await requireAdmin();
  const code = text(formData.get("code"));
  const to = text(formData.get("to")) as Step;
  if (!STEPS.includes(to)) redirect(`/admin/pesanan/${code}`);
  const order = await findOrder(code);

  let error: string | null = null;
  try {
```

```
    // Pesanan bayar online hanya boleh lunas lewat webhook gateway, supaya status tak bisa ditandai
tanpa uang masuk.
    const meta = await db.order.findUnique({ where: { id: order.id }, select: { paymentMethod: true,
courier: true, status: true } });
    if (to === "PAID" && meta?.paymentMethod === "GATEWAY") throw new OrderError("Pesanan bayar online
lunas otomatis begitu pembayaran dikonfirmasi gateway.");
    // Ongkir "estimasi" tidak menyebut kurir: admin memilihnya saat mengirim, agar pelanggan dapat
tautan lacak yang tepat.
    if (to === "SHIPPED" && meta && meta.status === "PACKED" && !detectCarrier(meta.courier)) {
      const carrier = carrierByCode(text(formData.get("carrier")));
      if (!carrier) throw new OrderError("Pilih kurir pengirim.");
      await db.order.update({ where: { id: order.id }, data: { courier: `${carrier.name}
(${meta.courier})`.slice(0, 60) } });
    }
    const r = await advanceOrder(order.id, to, admin.id, { trackingNumber: text(formData.get("resi"))
});
    const o = r.order;
    if (r.changed && o) {
      if (to === "PAID") after(() => emailOrderPaid(o));
      if (to === "SHIPPED") after(() => emailOrderShipped(o));
    }
  } catch (e) {
    if (!(e instanceof OrderError)) throw e;
    error = e.message;
  }
  redirect(error ? `/admin/pesanan/${code}?galat=${encodeURIComponent(error)}` :
`/admin/pesanan/${code}`);
}

export async function cancel(formData: FormData) {
  const admin = await requireAdmin();
  const code = text(formData.get("code"));
  const reason = text(formData.get("alasan")).trim().slice(0, 200) || "Dibatalkan admin";
  const order = await findOrder(code);


  const r = await cancelOrder(order.id, admin.id, reason);
  const o = r.order;
  if (r.changed && o) after(() => emailOrderCancelled(o, reason));
  redirect(`/admin/pesanan/${code}`);
}
```